

Supermarket Sales — Data Cleaning, Analysis & Visualization

Assignment: Data Cleaning, Analysis & Outliers Handling

Dataset: Supermarket Sales (1,000 transactions, Jan–Mar 2019)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams.update({'figure.dpi': 120, 'figure.facecolor': 'white'})
print('Libraries loaded successfully')
```

Section 1: Data Cleaning

```
# Load dataset
df = pd.read_csv('supermarket_sales.csv')
print(f'Shape: {df.shape}')
df.head()
```

```
# Dataset info
df.info()
```

```
# Descriptive statistics
df.describe().round(2)
```

```
# --- 1.1 Check Missing Values ---
print('Missing values per column:')
print(df.isnull().sum())
print(f'\nTotal missing: {df.isnull().sum().sum()}')
```

```
# --- 1.2 Check Duplicates ---
print(f'Duplicate rows: {df.duplicated().sum()}')
df.drop_duplicates(inplace=True)
print(f'Shape after deduplication: {df.shape}')
```

```
# --- 1.3 Fix Data Types ---
df['Date'] = pd.to_datetime(df['Date'])
df['Time'] = pd.to_datetime(df['Time'], format='%H:%M')

# Feature engineering
df['Hour'] = df['Time'].dt.hour
df['Month'] = df['Date'].dt.month_name()
df['Day'] = df['Date'].dt.day_name()

print('Date and Time columns converted to datetime')
print('New columns added: Hour, Month, Day')
```

```
# --- 1.4 Standardise Text Casing ---
cat_cols = ['Branch', 'City', 'Customer type', 'Gender', 'Product line', 'Payment']
for col in cat_cols:
    df[col] = df[col].str.strip().str.title()

print('Text columns normalised to Title Case')
df[cat_cols].head(3)
```

Section 2: Data Analysis

```
# --- 2.1 Univariate Analysis ---
num_cols = ['Unit price', 'Quantity', 'Tax 5%', 'Total', 'cogs', 'gross income', 'Rating']
print('Numeric Statistics:')
display(df[num_cols].describe().round(2))

print('\nCategorical Value Counts:')
for c in cat_cols:
    print(f'\n{c}:')
    print(df[c].value_counts())

# --- 2.2 Bivariate Analysis ---
print('Correlation Matrix:')
display(df[num_cols].corr().round(3))

print('\nAverage Total by Product Line:')
print(df.groupby('Product line')['Total'].mean().sort_values(ascending=False).round(2))

print('\nAverage Rating by Branch:')
print(df.groupby('Branch')['Rating'].mean().round(2))
```

Section 3: Outlier Handling

```
# --- 3A. Graphical: Box Plots ---
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
fig.suptitle('Box Plots - Outlier Detection', fontsize=14, fontweight='bold')
for i, col in enumerate(num_cols):
    ax = axes[0, i] if i < 4 else axes[1, i-4]
    ax.boxplot(df[col].dropna(), patch_artist=True,
               boxprops=dict(facecolor='#4C72B0', color='#333'),
               medianprops=dict(color='red', linewidth=2))
    ax.set_title(col)
axes[1, 3].set_visible(False)
plt.tight_layout()
plt.show()
```

```
# --- 3A. Graphical: Histograms ---
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
fig.suptitle('Histograms - Distribution Shape', fontsize=14, fontweight='bold')
for i, col in enumerate(num_cols):
    ax = axes[0, i] if i < 4 else axes[1, i-4]
    ax.hist(df[col].dropna(), bins=25, color='#4C72B0', edgecolor='white', alpha=0.85)
    ax.set_title(col)
axes[1, 3].set_visible(False)
plt.tight_layout()
plt.show()
```

```
# --- 3B. Statistical: Z-Score ---
z_scores = np.abs(stats.zscore(df[num_cols].dropna()))
z_outliers = (z_scores > 3).sum(axis=0)
print('Z-score outliers (|z| > 3) per column:')
for col, cnt in zip(num_cols, z_outliers):
    print(f' {col}: {cnt}')
```

```
# --- 3B. Statistical: IQR Method ---
print('IQR outliers per column:')
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    mask = (df[col] < lower) | (df[col] > upper)
    print(f' {col}: {mask.sum()} outliers bounds=[{lower:.2f}, {upper:.2f}]')

# Winsorize Rating (precautionary)
Q1_r = df['Rating'].quantile(0.25)
Q3_r = df['Rating'].quantile(0.75)
IQR_r = Q3_r - Q1_r
df['Rating'] = df['Rating'].clip(Q1_r - 1.5*IQR_r, Q3_r + 1.5*IQR_r)
print('\nRating winsorized (capped) using IQR bounds.')
```

Section 4: Data Visualization — Pandas Methods

```
# 4.1 Area Plot
daily = df.groupby('Date')['Total'].sum()
fig, ax = plt.subplots(figsize=(12, 4))
daily.plot(kind='area', ax=ax, color='#4C72B0', alpha=0.6)
ax.set_title('Daily Total Revenue - Area Plot')
ax.set_xlabel('Date'); ax.set_ylabel('Revenue ($)')
plt.tight_layout(); plt.show()
```

```
# 4.2 Bar Plot
product_sales = df.groupby('Product line')['Total'].sum().sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(10, 5))
product_sales.plot(kind='bar', ax=ax, color=sns.color_palette('muted', 6))
ax.set_title('Total Revenue by Product Line - Bar Plot')
plt.xticks(rotation=30, ha='right'); plt.tight_layout(); plt.show()
```

```
# 4.3 Horizontal Bar Plot
fig, ax = plt.subplots(figsize=(9, 5))
product_sales.sort_values().plot(kind='barh', ax=ax, color=sns.color_palette('muted', 6))
ax.set_title('Revenue by Product Line - Horizontal Bar')
plt.tight_layout(); plt.show()
```

```
# 4.4 Line Plot
fig, ax = plt.subplots(figsize=(12, 4))
daily.plot(kind='line', ax=ax, color='#DD4444', linewidth=1.5, marker='o', markersize=2)
ax.set_title('Daily Revenue - Line Plot')
plt.tight_layout(); plt.show()
```

```
# 4.5 Scatter Plot
fig, ax = plt.subplots(figsize=(8, 5))
df.plot(kind='scatter', x='Unit price', y='Total', alpha=0.4, color='#4C72B0', ax=ax)
ax.set_title('Unit Price vs Total - Scatter Plot')
plt.tight_layout(); plt.show()
```

```
# 4.6 Histogram
fig, ax = plt.subplots(figsize=(8, 4))
df['Total'].plot(kind='hist', bins=30, ax=ax, color='#4C72B0', edgecolor='white')
ax.set_title('Total Sales Distribution - Histogram')
plt.tight_layout(); plt.show()
```

```
# 4.7 KDE / Density Plot
fig, ax = plt.subplots(figsize=(8, 4))
df['Total'].plot(kind='kde', ax=ax, color='#DD4444', linewidth=2)
ax.set_title('Total Sales - KDE Density Plot')
plt.tight_layout(); plt.show()
```

```
# 4.8 Box Plot
fig, ax = plt.subplots(figsize=(10, 5))
df.boxplot(column='Total', by='Product line', ax=ax,
            medianprops=dict(color='red', linewidth=2))
ax.set_title('Total by Product Line - Box Plot')
plt.suptitle(''); plt.xticks(rotation=30, ha='right')
plt.tight_layout(); plt.show()
```

```
# 4.9 Hexbin Plot
fig, ax = plt.subplots(figsize=(8, 5))
df.plot(kind='hexbin', x='Unit price', y='Total', gridsize=20, cmap='Blues', ax=ax)
ax.set_title('Unit Price vs Total - Hexbin')
plt.tight_layout(); plt.show()
```

```
# 4.10 Pie Chart
fig, ax = plt.subplots(figsize=(7, 7))
df['Payment'].value_counts().plot(
    kind='pie', ax=ax, autopct='%1.1f%%',
    colors=sns.color_palette('muted', 3),
    startangle=140, wedgeprops=dict(edgecolor='white'))
ax.set_ylabel(''); ax.set_title('Payment Method Distribution - Pie Chart')
plt.tight_layout(); plt.show()
```

Section 5: Data Visualization — Seaborn Methods

```
# 5.1 KDE Plot (Seaborn)
fig, ax = plt.subplots(figsize=(8, 4))
sns.kdeplot(data=df, x='Total', fill=True, color='#4C72B0', ax=ax)
ax.set_title('Total Sales - Seaborn KDE Plot')
plt.tight_layout(); plt.show()
```

```
# 5.2 KDE + Rug Plot
fig, ax = plt.subplots(figsize=(8, 3))
sns.kdeplot(data=df, x='Rating', fill=True, alpha=0.4, ax=ax)
sns.rugplot(data=df, x='Rating', ax=ax, color='red', alpha=0.3)
ax.set_title('Rating Distribution - KDE + Rug Plot')
plt.tight_layout(); plt.show()
```

```
# 5.3 Joint Plot
g = sns.jointplot(data=df, x='Unit price', y='Total', kind='scatter',
                  height=6, color='#4C72B0', marginal_kws=dict(fill=True))
g.fig.suptitle('Joint Plot: Unit Price vs Total', y=1.01)
plt.show()
```

```
# 5.4 Pair Plot
pp = sns.pairplot(df[['Unit price', 'Quantity', 'Total', 'Rating', 'gross income']],
                  diag_kind='kde', plot_kws={'alpha': 0.4, 's': 15},
                  diag_kws={'fill': True})
pp.fig.suptitle('Pair Plot - Key Variables', y=1.01)
plt.show()
```

```
# 5.5 Factor Plot (catplot)
g = sns.catplot(data=df, x='Branch', y='Total', hue='Customer type',
                kind='bar', palette='muted', height=5, aspect=1.4)
g.fig.suptitle('Avg Total by Branch & Customer Type - Factor Plot', y=1.02)
plt.show()
```

```
# 5.6 Box Plot (Seaborn)
fig, ax = plt.subplots(figsize=(12, 5))
sns.boxplot(data=df, x='Product line', y='Total', palette='muted', ax=ax)
ax.set_title('Total by Product Line - Seaborn Box Plot')
plt.xticks(rotation=30, ha='right'); plt.tight_layout(); plt.show()
```

```
# 5.7 Violin Plot
fig, ax = plt.subplots(figsize=(12, 5))
sns.violinplot(data=df, x='Product line', y='Total', palette='muted', ax=ax)
ax.set_title('Total by Product Line - Violin Plot')
plt.xticks(rotation=30, ha='right'); plt.tight_layout(); plt.show()
```

```
# 5.8 Strip Plot
fig, ax = plt.subplots(figsize=(12, 5))
sns.stripplot(data=df, x='Product line', y='Rating',
              palette='muted', jitter=True, alpha=0.5, ax=ax)
ax.set_title('Rating by Product Line - Strip Plot')
plt.xticks(rotation=30, ha='right'); plt.tight_layout(); plt.show()
```

```
# 5.9 Swarm Plot
sample = df.sample(200, random_state=42)
fig, ax = plt.subplots(figsize=(10, 5))
sns.swarmplot(data=sample, x='Branch', y='Total', hue='Gender',
              palette='muted', ax=ax, size=4)
ax.set_title('Total by Branch & Gender - Swarm Plot (200-row sample)')
plt.tight_layout(); plt.show()
```

```
# 5.10 Bar Plot (Seaborn)
fig, ax = plt.subplots(figsize=(12, 5))
sns.barplot(data=df, x='Product line', y='Total', hue='Gender',
            palette='muted', ax=ax, errorbar='sd')
ax.set_title('Avg Total by Product Line & Gender - Seaborn Bar Plot')
plt.xticks(rotation=30, ha='right'); plt.tight_layout(); plt.show()
```

```
# 5.11 Count Plot
fig, ax = plt.subplots(figsize=(12, 4))
sns.countplot(data=df, x='Product line', hue='Customer type',
               palette='muted', ax=ax)
ax.set_title('Count by Product Line & Customer Type - Count Plot')
plt.xticks(rotation=30, ha='right'); plt.tight_layout(); plt.show()
```

```
# 5.12 Heatmap - Correlation Matrix
fig, ax = plt.subplots(figsize=(9, 7))
corr = df[num_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', linewidths=0.5, ax=ax, vmin=-1, vmax=1)
ax.set_title('Correlation Matrix - Heatmap')
plt.tight_layout(); plt.show()
```

Summary

KPI	Value	
Total Revenue	322,	322.97
	967	
	AvgTransaction	
Top Product Line	Food & Beverages	
Best Branch	C (Naypyitaw)	
Avg Rating	6.97/10	
Most Popular Payment	Ewallet	

Key Findings:

- Dataset was clean — no missing values or duplicates
- 9 IQR outliers in Total/cogs — legitimate high-value transactions, retained
- Rating is uncorrelated with spending — service quality is independent of purchase size
- Branch C outperforms by revenue; payment methods are evenly split